Control Flow, Functions & Program Structure

Session 08 practical workbook · Python 3 · VS Code · Sales order processor

In Session 07 every program ran once, top to bottom, the same way every time. Today the program learns to **choose** (conditions), **repeat** (loops), **reuse** (functions), **recover** from bad input (exceptions) and **coordinate** everything from one place (`main()`).

Each idea is introduced with a small, everyday business case, then applied to the sales order from Session 07. The session ends with one complete program: a Sales Order Processor. Every example was run on Python 3; the output shown is the real output.

Part	Topic	Business case used to explain it
1	Foundations	Is this a large order? A comparison gives True or False
2	Decisions: `if`, `elif`, `else`	Free delivery, large orders, order bands, loan pre-approval
3	Loops: `for` and `while`	Instalment plans, till cash-up, ATM PIN attempts, paying off a loan
4	Functions	VAT calculation, sales commission
5	Composition and scope	Building an order total from small functions
6	Exceptions: `try` / `except`	A cashier types "four" instead of 4
7	Program structure	Organising the order program with `main()`
8	Build the Sales Order Processor	Everything together in one script

Session model. Five ideas, each with one job:

Condition: choose → **Loop: repeat** → **Function: reuse** → **Exception: recover** → **main(): coordinate**

Lesson plan (3 hours)

Use the slides to introduce each idea, then move to VS Code. Each block follows the same rhythm: one business case, live code, a prediction, then learners try it.

Time	Block	You do (trainer)	Learners do	Slides
0–15	Bridge and Part 1	Show the Session 07 program that knows the answer but can only print it. Explain indentation as block ownership.	Predict True/False for two totals.	1–7
15–40	Part 2 · Decisions	Free delivery → large order → order bands. Run the branch-order defect and let them find it.	Try it yourself 2.	8–13
40–70	Part 3 · Loops	`for` with an instalment plan, ATM PIN with `continue`/`break`, `while` with a loan payoff.	Trace values; Try it yourself 3.	14–22
70–95	Part 4 · Functions	VAT function on the board. Contrast `print` with `return`.	Try it yourself 4.	23–30
95–105	Break			31
105–120	Part 5 · Composition and scope	Trace 945.0 through three functions. Show the missing-return defect.	Try it yourself 5.	32–38
120–140	Part 6 · Exceptions	"four" versus "-2": exception versus ordinary rule. Build `read_positive_int` live.	Try it yourself 6.	39–49
140–150	Part 7 · Structure	Script order, `process_order()`, `main()`, the guard.	Try it yourself 7.	50–54
150–175	Part 8 · Build	Stop lecturing; coach. Check 945.0 and 1181.25.	Complete the starter file; test every path.	55–62
175–180	Close	Concept contrasts and the eight questions.	Answer aloud.	63–67

Before class

- ☐ Learners have VS Code with the Python extension, as in Session 07.
- ☐ A new folder `s08_control_flow` opened in VS Code.
- ☐ Starter file `sales_order_starter.py` from the class bundle copied into it.
- ☐ Know how to stop a running program: click in the terminal and press **Ctrl+C**. You will need it if a loop never ends.

Teaching moves for this session

Move	How it sounds in class
Trace before running	"Which branch will run? What values will the loop variable take? Write them down."
Point at the indentation	"Which lines belong to this `if`? How can you tell?"
Name the exit	"How does this loop end? Which line changes the condition?"
Returned, not printed	"Did this function hand a value back, or only show it?"
Exception or rule?	"Did the conversion fail, or did it succeed and break a business rule?"

Before you start: where Session 07 left us

This is how the Session 07 program ended. It works out the right answer, but it can only display it:

bridge.py

```
final_total = 945.0
large_order_threshold = 950.0

is_large_order = final_total >= large_order_threshold
print("Final total:", final_total)
print("Large order?", is_large_order)
```

OUTPUT

```
Final total: 945.0
Large order? False
```

The program *knows* the order is not large, yet it cannot act on that. It cannot offer a discount for large orders, ask again after a typing mistake, or process a second order. Today's session asks four new questions about every program: **which** steps should run, **how many times**, **which function** owns each job, and **what happens** when input is wrong.

> **In VS Code:** VS Code adds the indentation for you: after you type a line ending in `:` and press Enter, the next line is indented automatically. Press **Tab** to indent and **Shift+Tab** to move back. Python expects 4 spaces.

Part 1 Foundations

Three facts from Session 07 make everything today possible.

1. A comparison produces True or False

p1_compare.py

```
print(945.0 >= 950.0)
print(1181.25 >= 950.0)
```

OUTPUT

```
False
True
```

The comparison creates the Boolean. Today's new tools (`if`, `while`) do not create it; they **use** it to decide what runs next.

2. Indentation shows which lines belong together

p1_blocks.py

```
final_total = 1181.25

if final_total >= 950.0:
    print("Large-order path")   # indented: inside the if
print("Order checked")          # not indented: outside it
```

OUTPUT

```
Large-order path
Order checked
```

A line ending in `:` introduces a **block**. The indented lines below it belong to that block. This rule is the same for decisions, loops, functions and exception handlers, so learn to read it now. Forgetting the indent is an error:

p1_indent_error.py

```
final_total = 1181.25
if final_total >= 950.0:
print("Large-order path")
```

ERROR

```
  File "C:\Users\student\s07_python_basics\p1_indent_error.py", line 3
    print("Large-order path")
    ^^^^^
IndentationError: expected an indented block after 'if' statement on line 2
```

3. input() gives text, and defining is not running

`input()` still returns a string, so `int("four")` still fails with a `ValueError`. Today we learn to recover from that. We also meet `def`, which creates a function without running it; the code inside runs only when the function is called.

Part 2 Decisions: choosing a path

Businesses run on rules: *if* the order is big enough, delivery is free; *if* income is too low, the loan is referred. An `if` statement turns a rule like that into code.

if: run a block only when the condition is true

> **Business case:** An online shop offers free delivery on orders of R500 or more.

p2_if.py

```
order_total = 620.0

if order_total >= 500:
    print("Free delivery applied")

print("Checkout complete")
```

OUTPUT

```
Free delivery applied
Checkout complete
```

Read it like a sentence: *if this condition is true, do this work*. Change `order_total` to `320.0` and run again: the indented line is skipped, but `Checkout complete` still prints because it is not inside the `if`.

if / else: choose between two paths

> **Business case:** Orders of 950.0 or more are large orders and get special handling. Every other order is standard.

PREDICT FIRST

With `final_total = 945.0`, which message appears?

p2_if_else.py

```
final_total = 945.0
large_order_threshold = 950.0

if final_total >= large_order_threshold:
    print("Large-order path")
else:
    print("Standard-order path")
```

OUTPUT

```
Standard-order path
```

`945.0 >= 950.0` is `False`, so the `if` block is skipped and the `else` block runs. `else` has no condition of its own: it is simply the path taken when the condition was not true. Exactly one of the two blocks always runs.

> **Watch out:** `if` does not change the Boolean. The comparison already produced `False`; `if / else` only chooses what to do with it.

if / elif / else: more than two paths

> **Business case:** The sales team bands orders: 1500 or more is *priority-large*, 950 or more is *large*, anything else is *standard*.

PREDICT FIRST

For `final_total = 1181.25`, which band is printed?

p2_elif.py

```
final_total = 1181.25

if final_total >= 1500.0:
    order_band = "priority-large"
elif final_total >= 950.0:
    order_band = "large"
else:
    order_band = "standard"

print("Order band:", order_band)
```

OUTPUT

```
Order band: large
```

Check	Result	What Python does
`1181.25 >= 1500.0`	False	skip this branch, keep checking
`1181.25 >= 950.0`	True	run this branch, then **stop checking**
`else`	never reached	

Rule: Python checks from the top and runs the **first** branch whose condition is true. Everything after it is skipped.

A deliberate defect: branch order matters

Someone adds a *review* band for orders of 500 or more, but puts it first:

p2_defect.py

```
final_total = 1181.25

if final_total >= 500.0:
    order_band = "review"
elif final_total >= 950.0:
    order_band = "large"
else:
    order_band = "standard"

print("Observed label:", order_band)
print("Expected label: large")
```

OUTPUT

```
Observed label: review
Expected label: large
```

No error, but a wrong answer. `1181.25 >= 500.0` is already True, so the first branch wins and the `large` check is never reached. **Fix:** check the higher, more specific threshold first:

p2_fixed.py

```
final_total = 1181.25

if final_total >= 950.0:
    order_band = "large"
elif final_total >= 500.0:
    order_band = "review"
else:
    order_band = "standard"

print("Corrected label:", order_band)
```

OUTPUT

```
Corrected label: large
```

Combining conditions

> **Business case:** A bank pre-approves a personal loan only if monthly income is at least R15 000 **and** the credit score is at least 650.

p2_and.py

```
monthly_income = 18000.0
credit_score = 640

if monthly_income >= 15000 and credit_score >= 650:
    print("Pre-approved")
else:
    print("Refer to a consultant")
```

OUTPUT

```
Refer to a consultant
```

`and` needs both sides to be True; `or` needs at least one. The income passes but the score does not, so the application is referred.

> TRY IT YOURSELF
>
> **(a) Loyalty tiers.** A retailer gives customers a tier based on their annual spend: **Gold** for 20 000 or more, **Silver** for 10 000 or more, otherwise **Bronze**. Write the `if / elif / else` and test it with 25 000, 12 500 and 4 000.
>
> **(b) Delivery fee.** Set `delivery_fee` to `0.0` if the order is 500 or more, otherwise `60.0`. Print the order total plus the delivery fee for an order of 450.0.

Part 3 Loops: repeating a path

Some work has to happen more than once: one line per instalment, one count per till, another try at a PIN. A **loop** repeats a block.

Choose the loop by why it repeats

Loop	Use it when	Business example	When does it stop?
`for`	the number of repetitions is known in advance	6 monthly instalments; 3 PIN attempts	built into the range
`while`	repetition continues while a condition stays true	keep paying until the loan balance is 0	when the condition becomes False

for and range()

PREDICT FIRST

Before running, write down the values `attempt` will take.

p3_range.py

```
for attempt in range(1, 4):
    print("Attempt number:", attempt)
```

OUTPUT

```
Attempt number: 1
Attempt number: 2
Attempt number: 3
```

`range(1, 4)` starts at 1 and stops **before** 4, so the body runs exactly three times with `attempt` set to 1, 2 and 3. The variable after `for` changes automatically on every pass.

> **Business case:** A customer buys a laptop for R6 000 on a 6-month instalment plan. Print the balance after each payment.

p3_instalments.py

```
laptop_price = 6000.0
months = 6
instalment = laptop_price / months
balance = laptop_price

for month in range(1, months + 1):
    balance = balance - instalment
    print(f"Month {month}: paid R{instalment:.2f}, balance R{balance:.2f}")

print("Plan complete")
```

OUTPUT

```
Month 1: paid R1000.00, balance R5000.00
Month 2: paid R1000.00, balance R4000.00
Month 3: paid R1000.00, balance R3000.00
Month 4: paid R1000.00, balance R2000.00
Month 5: paid R1000.00, balance R1000.00
Month 6: paid R1000.00, balance R0.00
Plan complete
```

`range(1, months + 1)` gives 1 to 6. Each pass updates `balance`, so the next pass starts from the new value. `Plan complete` is not indented, so it runs once, after the loop.

The running-total pattern

> **Business case:** At closing time a supermarket manager enters the takings from each of its 3 tills and needs the total.

p3_cashup.py

```
total_takings = 0.0

for till in range(1, 4):
    amount = float(input(f"Takings for till {till}: "))
    total_takings = total_takings + amount

print(f"Total takings: R{total_takings:.2f}")
```

TERMINAL

```
> python p3_cashup.py
Takings for till 1: 4250.50
Takings for till 2: 3890.00
Takings for till 3: 5120.25
Total takings: R13260.75
```

Start the total at 0 **before** the loop, add to it **inside** the loop, and report it **after** the loop. You will use this pattern constantly in data work.

continue and break

> **Business case:** An ATM allows at most three attempts to enter a PIN. A blank entry does not count as a real attempt at the PIN. A correct PIN ends the loop at once.

p3_pin.py

```
correct_pin = "4321"
pin_accepted = False

for attempt in range(1, 4):
    pin = input(f"Attempt {attempt}/3 - enter your PIN: ").strip()

    if pin == "":
        print("PIN cannot be blank. Try again.")
        continue

    if pin == correct_pin:
        print("PIN accepted. Welcome.")
        pin_accepted = True
        break

    print("Incorrect PIN.")

if not pin_accepted:
    print("Card blocked after three attempts.")
```

TERMINAL

```
> python p3_pin.py
Attempt 1/3 - enter your PIN:
PIN cannot be blank. Try again.
Attempt 2/3 - enter your PIN: 4321
PIN accepted. Welcome.
```

TERMINAL · ANOTHER RUN

```
> python p3_pin.py
Attempt 1/3 - enter your PIN: 1111
Incorrect PIN.
Attempt 2/3 - enter your PIN: 2222
Incorrect PIN.
Attempt 3/3 - enter your PIN: 3333
Incorrect PIN.
Card blocked after three attempts.
```

Keyword	What it does	In the PIN example
`continue`	skips the rest of **this** pass and goes to the next one	blank PIN: skip to the next attempt
`break`	leaves the loop completely, even if passes remain	correct PIN: stop asking

`pin_accepted` remembers whether the loop ended with `break`. After the loop, `if not pin_accepted` decides whether to block the card.

while: repeat until something changes

> **Business case:** A customer owes R5 000 and pays R1 250 a month. How many months until the loan is settled?

p3_while_loan.py

```
balance = 5000.0
monthly_payment = 1250.0
months = 0

while balance > 0:
    balance = balance - monthly_payment
    months = months + 1
    print(f"After month {months}: balance R{balance:.2f}")

print(f"Loan settled in {months} months.")
```

OUTPUT

```
After month 1: balance R3750.00
After month 2: balance R2500.00
After month 3: balance R1250.00
After month 4: balance R0.00
Loan settled in 4 months.
```

There is no fixed number of repetitions. Python checks `balance > 0` before every pass; once the balance reaches 0 the condition is False and the loop ends. The same shape keeps asking for input until it is valid:

p3_while_name.py

```
customer_name = ""

while customer_name == "":
    customer_name = input("Customer name: ").strip()
    if customer_name == "":
        print("Customer name cannot be blank.")

print("Accepted customer:", customer_name)
```

TERMINAL

```
> python p3_while_name.py
Customer name:
Customer name cannot be blank.
Customer name: Maya Patel
Accepted customer: Maya Patel
```

Every while loop needs a reachable exit

p3_safe.py

```
counter = 1

while counter <= 3:
    print("Counter:", counter)
    counter = counter + 1
```

OUTPUT

```
Counter: 1
Counter: 2
Counter: 3
```

Now imagine the line `counter = counter + 1` is missing. `counter` stays 1, the condition stays True, and the loop prints 1 forever. Before running any `while` loop, ask: **which value controls the condition, and where does it change?**

> **Watch out:** If a program never ends, click in the VS Code terminal and press **Ctrl+C** to stop it. An infinite loop is a logic mistake to fix, not something `try / except` can hide.

Later you will see `while True:`. Its condition never becomes False, so it is only safe when the body has a clear exit such as `break` or `return`. Part 6 uses exactly this pattern.

> TRY IT YOURSELF
>
> **(a) Savings goal (while).** A customer saves R1 500 a month towards a R10 000 goal. Use a `while` loop to print the savings after each month and how many months it takes.
>
> **(b) Weekly sales (for).** Use `for` and `range` to ask for the sales of 4 weeks, then print the total and the average (total ÷ 4).

Part 4 Functions: naming a responsibility

When a calculation is needed in many places, writing it out every time invites mistakes. A **function** gives the job a name, takes inputs, and hands back a result.

> **Business case:** Every invoice needs VAT at 15%. Instead of writing `amount * 0.15` all over the program, give the job one name.

p4_vat.py

```
def calculate_vat(amount):
    vat = amount * 0.15
    return vat

print(calculate_vat(200.0))
print(calculate_vat(1250.0))
```

OUTPUT

```
30.0
187.5
```

Anatomy of a function

p4_anatomy.py

```
def calculate_subtotal(unit_price, quantity):  # definition
    subtotal = unit_price * quantity           # body: the work
    return subtotal                            # hand the value back

subtotal = calculate_subtotal(250.0, 4)        # call
print(subtotal)
```

OUTPUT

```
1000.0
```

Part	Meaning
`def calculate_subtotal`	defines a function and names its responsibility
`(unit_price, quantity)`	**parameters**: names for the inputs
indented body	the work; it runs only when the function is **called**
`return subtotal`	sends the value back to the caller and leaves the function
`calculate_subtotal(250.0, 4)`	the **call**; `250.0` and `4` are the **arguments**

Parameters and arguments

A **parameter** is the name in the definition. An **argument** is the value you pass in the call. One definition works with many arguments:

p4_params.py

```
def calculate_subtotal(unit_price, quantity):
    return unit_price * quantity

print(calculate_subtotal(250.0, 4))
print(calculate_subtotal(125.0, 2))
```

OUTPUT

```
1000.0
250.0
```

print() displays; return hands a value back

PREDICT FIRST

What will `printed_result` and `returned_result` hold?

p4_print_return.py

```
def display_subtotal(unit_price, quantity):
    subtotal = unit_price * quantity
    print("Displayed subtotal:", subtotal)

def return_subtotal(unit_price, quantity):
    subtotal = unit_price * quantity
    return subtotal

printed_result = display_subtotal(250.0, 4)
returned_result = return_subtotal(250.0, 4)

print("From display_subtotal:", printed_result)
print("From return_subtotal:", returned_result)
```

OUTPUT

```
Displayed subtotal: 1000.0
From display_subtotal: None
From return_subtotal: 1000.0
```

`display_subtotal` shows the number but gives nothing back, so the caller receives `None` (Python's word for "no value"). Only a returned value can be stored and used in the next calculation.

Default parameter values

> **Business case:** Sales agents earn 5% commission. Senior agents earn 7.5%.

p4_default.py

```
def calculate_commission(sales, rate=0.05):
    return sales * rate

print(calculate_commission(40000.0))
print(calculate_commission(40000.0, 0.075))
```

OUTPUT

```
2000.0
3000.0
```

`rate=0.05` is used when the caller leaves it out, and replaced when the caller supplies a value. The deck's order classifier works the same way:

p4_classify.py

```
def classify_order(final_total, threshold=950.0):
    if final_total >= threshold:
        return "large"
    else:
        return "standard"

print("945.0 ->", classify_order(945.0))
print("1181.25 ->", classify_order(1181.25))
print("945.0 with threshold 900.0 ->", classify_order(945.0, 900.0))
```

OUTPUT

```
945.0 -> standard
1181.25 -> large
945.0 with threshold 900.0 -> large
```

Three ways to leave

Keyword	Leaves	Goes to
`continue`	the current pass of a loop	the next pass
`break`	the whole loop	the first line after the loop
`return`	the whole function (and any loop inside it)	back to the caller, with a value

TRY IT YOURSELF

(a) Write `apply_discount(subtotal, discount_pct)` that **returns** the discounted subtotal. `apply_discount(1000.0, 10.0)` should give `900.0`.

(b) Write `delivery_fee(order_total, free_threshold=500.0)` that returns `0.0` when the order reaches the threshold and `60.0` otherwise. Test it with 620.0, 450.0, and 450.0 with a threshold of 400.0.

Part 5 Composition and scope

Small functions become a program when one function uses the results of others. This is **composition**.

Functions calling functions

p5_compose.py

```
def calculate_subtotal(unit_price, quantity):
    return unit_price * quantity

def apply_discount(subtotal, discount_pct):
    discount_amount = subtotal * discount_pct / 100
    return subtotal - discount_amount

def apply_tax(discounted_subtotal, tax_pct):
    tax_amount = discounted_subtotal * tax_pct / 100
    return discounted_subtotal + tax_amount

def calculate_final_total(unit_price, quantity, discount_pct, tax_pct):
    subtotal = calculate_subtotal(unit_price, quantity)
    discounted_subtotal = apply_discount(subtotal, discount_pct)
    final_total = apply_tax(discounted_subtotal, tax_pct)
    return final_total

print(calculate_final_total(250.0, 4, 10.0, 5.0))
print(calculate_final_total(250.0, 5, 10.0, 5.0))
```

OUTPUT

```
945.0
1181.25
```

Value	Quantity 4	Quantity 5
`calculate_subtotal`	1000.0	1250.0
`apply_discount` (10%)	900.0	1125.0
`apply_tax` (5%) = final total	945.0	1181.25
`classify_order`	standard	large

Each function does one job and **returns** its result, so the next function can use it. If a total is wrong, you can test each small function on its own to find the faulty one.

Local names stay inside the function

p5_scope.py

```
def demonstrate_local_scope():
    local_status = "validated inside the function"
    print("Inside function:", local_status)

demonstrate_local_scope()
print("Outside function:", local_status)
```

OUTPUT

```
Inside function: validated inside the function
```

ERROR

```
Traceback (most recent call last):
  File "C:\Users\student\s07_python_basics\p5_scope.py", line 6, in <module>
    print("Outside function:", local_status)
                               ^^^^^^^^^^^^
NameError: name 'local_status' is not defined
```

A variable created inside a function is **local**: it exists only while that function runs. To get a value out, `return` it.

Constants at the top, data through parameters

p5_constants.py

```
LARGE_ORDER_THRESHOLD = 950.0
VAT_RATE = 0.15

def classify_order(final_total, threshold=LARGE_ORDER_THRESHOLD):
    if final_total >= threshold:
        return "large"
    return "standard"

print(classify_order(1181.25))
print(VAT_RATE * 100, "% VAT")
```

OUTPUT

```
large
15.0 % VAT
```

Business settings that rarely change (thresholds, rates) live at the top of the script in UPPERCASE names, a convention meaning *this is a constant*. The function still receives the value through a parameter, so it is clear what it depends on.

A deliberate defect: the missing return

p5_missing_return.py

```
def broken_subtotal(unit_price, quantity):
    subtotal = unit_price * quantity
    # the value is never returned

result = broken_subtotal(250.0, 4)
print("Result received by caller:", result)
```

OUTPUT

```
Result received by caller: None
```

The function calculated 1000.0 and then threw it away. Without `return`, the caller gets `None`. If a later line does maths with it you will see `TypeError: unsupported operand type(s) ... 'NoneType'`, which almost always means a missing `return`.

> **TRY IT YOURSELF**
> Predict the label this code returns, then run it. Which function calls which?

p5_try_label.py

```
def clean_text(text):
    return text.strip()

def build_order_label(customer_name, product_name):
    clean_customer = clean_text(customer_name).title()
    clean_product = clean_text(product_name).title()
    return clean_customer + " - " + clean_product

print(build_order_label("  maya patel ", " analytics starter kit "))
```

Part 6 Exceptions: recovering from expected failure

> **Business case:** A cashier is asked for the quantity and types `four`. Right now the whole program crashes. The business wants it to say what went wrong and ask again.

p6_problem.py

```
raw_quantity = input("Quantity: ")
quantity = int(raw_quantity)
print("Quantity:", quantity)
```

TERMINAL

```
> python p6_problem.py
Quantity: four
```

ERROR

```
Traceback (most recent call last):
  File "C:\Users\student\s07_python_basics\p6_problem.py", line 2, in <module>
    quantity = int(raw_quantity)
ValueError: invalid literal for int() with base 10: 'four'
```

Two different problems

The user types	What happens	Handled by
`four`	`int("four")` fails with a ValueError	`try / except` (exception path)
`-2`	`int("-2")` works and gives -2, but a quantity must be above 0	an ordinary `if` (business rule)
`5`	conversion works and the rule passes	accepted

`-2` is a perfectly good integer. The problem is the business rule, so it is checked with `if`, not with an exception.

try and a specific except

p6_try.py

```
raw_quantity = "four"

try:
    quantity = int(raw_quantity)
except ValueError:
    print("Could not convert the quantity to a whole number.")

print("The program is still running.")
```

OUTPUT

```
Could not convert the quantity to a whole number.
The program is still running.
```

Python tries the risky line. If it raises a `ValueError`, Python jumps into the matching `except` block instead of stopping the program.

> **Watch out:** Keep the `try` block small (just the risky line) and name the exception. A bare `except:` catches everything, including your own typos, and hides real bugs.

else and finally

p6_else_finally.py

```
raw_quantity = "four"

try:
    quantity = int(raw_quantity)
except ValueError:
    print("Could not convert the quantity.")
else:
    print("Converted quantity:", quantity)
finally:
    print("Conversion attempt finished.")
```

OUTPUT

```
Could not convert the quantity.
Conversion attempt finished.
```

TERMINAL · ANOTHER RUN

```
> python p6_else_finally.py
Converted quantity: 5
Conversion attempt finished.
```

Second run: the same code with `raw_quantity = "5"`.

Block	Runs when
`try`	always attempted
`except ValueError`	the conversion failed
`else`	the conversion succeeded
`finally`	always, either way

A validated input helper

Now combine a loop, an exception and a condition into one reusable function. It keeps asking until it gets a positive whole number:

p6_read_positive_int.py

```
def read_positive_int(prompt):
    while True:
        raw_value = input(prompt)

        try:
            value = int(raw_value)
        except ValueError:
            print("Please enter a whole number.")
            continue
        else:
            if value <= 0:
                print("Value must be greater than 0.")
                continue

            return value

quantity = read_positive_int("Quantity: ")
print("Accepted quantity:", quantity)
```

TERMINAL

```
> python p6_read_positive_int.py
Quantity: four
Please enter a whole number.
Quantity: 0
Value must be greater than 0.
Quantity: -2
Value must be greater than 0.
Quantity: 5
Accepted quantity: 5
```

Input	Path
`four`	`int()` fails → `except` → message → `continue` (ask again)
`0`, `-2`	`int()` works → `if value <= 0` is True → message → `continue`
`5`	`int()` works → rule passes → `return 5` leaves the function, ending the loop

`while True` is safe here because `return` is a clear, reachable exit.

> **Watch out:** Exceptions handle **expected** runtime failures such as bad input. They do not fix programming mistakes: a missing `return`, a wrong branch order, a loop that never ends or a wrong formula must be debugged, not wrapped in `try`.

> TRY IT YOURSELF
>
> Write `read_percentage(prompt)` with the same shape, but using `float()`. It must reject text that is not a number, values below 0 ("Percentage cannot be below 0.") and values above 100 ("Percentage cannot be above 100."). Test with `ten`, `-5`, `120` and `10`. For each failing input, say whether it took the exception path or an ordinary condition.

Part 7 Program structure

A notebook lets you run cells in any order. A script is read by Python from top to bottom, every time, so it needs a clear layout.

A clean script has five sections

Order	Section	Example
1	Constants	`LARGE_ORDER_THRESHOLD = 950.0`
2	Helper functions	`read_...`, `calculate_...`, `apply_...`, `classify_order`, `display_summary`
3	`process_order()`	handles one complete order
4	`main()`	repeats orders until the user stops
5	Entry guard	`if __name__ == "__main__":` then `main()`

Python must see a `def` before the function is called. Defining a function does not run it, which is why all the definitions can sit above the one line at the bottom that starts the program.

process_order() coordinates; it does not calculate

```
def process_order():
    customer_name = read_required_text("Customer name: ")
    product_name = read_required_text("Product name: ")
    unit_price = read_positive_number("Unit price: ")
    quantity = read_positive_int("Quantity: ")
    discount_pct = read_percentage("Discount %: ")
    tax_pct = read_percentage("Tax %: ")

    final_total = calculate_final_total(unit_price, quantity, discount_pct, tax_pct)
    classification = classify_order(final_total)

    display_summary(customer_name, product_name, unit_price, quantity,
                    discount_pct, tax_pct, final_total, classification)
```

Each line names a job done somewhere else. If the total is wrong you look in the calculation helpers; if the prompt is wrong you look in the input helpers.

main(): the one place the program is run from

```
def main():
    print("Sales Order Validation & Processing Program")

    while True:
        process_order()

        again = input("Process another order? (y/n): ").strip().lower()
        if again != "y":
            print("Program finished.")
            break
```

`while True` repeats whole orders. Anything other than `y` prints the closing message and `break` ends the loop cleanly.

The __main__ guard

p7_guard.py

```
print("Current __name__ value:", __name__)

if __name__ == "__main__":
    print("This file was run directly, so main() would start here.")
```

OUTPUT

```
Current __name__ value: __main__
This file was run directly, so main() would start here.
```

When you run a file directly, Python sets `__name__` to `"__main__"`, the condition is True and `main()` starts. When another file imports this one later (Session 10), `__name__` is different and the program does not start by itself. It is an ordinary `if`, not magic.

> TRY IT YOURSELF
>
> These five pieces of a script are in the wrong order. Put them in the right order: (A) `def main(): ...` (B) `if __name__ == "__main__": main()` (C) `TAX_RATE = 0.15` (D) `def process_order(): ...` (E) `def apply_tax(...): ...`

Part 8 Build the Sales Order Processor

> **Trainer note:** From here learners build and you coach. Circulate during steps 3 and 6, where composition and coordination mistakes usually appear. If a total is wrong, test the small functions one at a time.

A small business wants a command-line program that processes one sales order at a time, shows a summary, then asks whether to continue.

#	Requirement	Tool from today
1	Customer and product names must not be blank	`while` loop · `if` · `continue` · `return`
2	Unit price must be a positive number	`try / except ValueError` · `if`
3	Quantity must be a positive whole number	`try / except ValueError` · `if`
4	Discount and tax must be between 0 and 100	`try / except ValueError` · `if / elif`
5	Final total built from small calculation steps	functions · `return` · composition
6	Order is large at or above 950.0	`if / else` · default parameter
7	Readable order summary	a display function
8	Offer another order	`main()` · `while True` · `break`
9	Start only when the file is run directly	`__main__` guard

Checkpoints every correct program must produce

	Quantity 4	Quantity 5
Unit price	250.0	250.0
Subtotal	1000.0	1250.0
After 10% discount	900.0	1125.0
After 5% tax (final total)	945.0	1181.25
Classification	standard	large

The two orders sit either side of the 950.0 threshold, so one tests the standard branch and the other the large branch.

Build it in seven small steps

Open `sales_order_starter.py`, save it as `sales_order_processor.py`, and complete it in this order. Test each step before moving on.

Step	You build	Test it with
1	`read_required_text(prompt)`	Enter on a blank value once, then Maya Patel
2	`read_positive_number`, `read_positive_int`, `read_percentage`	abc, four, 0, -2, 120 and valid values
3	`apply_tax` and `calculate_final_total`	the checkpoints 945.0 and 1181.25
4	`classify_order(final_total, threshold=...)`	945.0 gives standard; 1181.25 gives large
5	`display_summary(...)`	a readable summary, no calculation inside
6	`process_order()`	one full order from prompts to summary
7	`main()` with `break`, then the guard	process two orders, then choose n

A quick way to test step 3 without typing anything is `assert`, which stops with an error if a statement is not True:

```
assert calculate_final_total(250.0, 4, 10.0, 5.0) == 945.0
assert calculate_final_total(250.0, 5, 10.0, 5.0) == 1181.25
print("Both checkpoints pass")
```

Test every path

Test	What should happen
Maya Patel · 250.0 · quantity 4 · discount 10 · tax 5	final total 945.0; standard
Same values, quantity 5	final total 1181.25; large
Quantity typed as `four`	ValueError handled; asks again
Quantity 0 or negative	conversion works; an ordinary condition rejects it
Percentage below 0 or above 100	an ordinary condition rejects it
Blank customer name	the loop asks again
Answer `n` to another order	the program ends cleanly

Reference run

A finished program, run with every kind of bad input along the way. Your wording may differ; the flow and the totals should match.

TERMINAL

```
> python sales_order_processor.py
Sales Order Validation & Processing Program
Customer name:
This field cannot be blank.
Customer name: Maya Patel
Product name: Analytics Starter Kit
Unit price: abc
Please enter a number.
Unit price: 250.0
Quantity: four
Please enter a whole number.
Quantity: 0
Value must be greater than 0.
Quantity: 4
Discount %: 120
Percentage cannot be above 100.
Discount %: 10
Tax %: 5

--- Order Summary ---
Customer: Maya Patel
Product: Analytics Starter Kit
Unit price: 250.0
Quantity: 4
Discount %: 10.0
Tax %: 5.0
Final total: 945.0
Classification: standard
Process another order? (y/n): y
Customer name: Maya Patel
Product name: Analytics Starter Kit
Unit price: 250
Quantity: 5
Discount %: 10
Tax %: 5

--- Order Summary ---
Customer: Maya Patel
Product: Analytics Starter Kit
Unit price: 250.0
Quantity: 5
Discount %: 10.0
Tax %: 5.0
Final total: 1181.25
Classification: large
Process another order? (y/n): n
Program finished.
```

Ask learners to point to the line of their own code that produced each line of this transcript.

Final clean run

- ☐ Close any open Python terminals and save the file.

- ☐ Run it with ▶ from a fresh terminal.
- ☐ Check both totals: 945.0 and 1181.25.
- ☐ Test one invalid conversion (`four`) and one valid-but-out-of-range value (`120`).
- ☐ Answer `n` and confirm the program finishes cleanly.

Close: the distinctions that matter

Pair	Key distinction
comparison vs `if`	a comparison produces a Boolean; `if` uses it
`if` vs `elif`	`elif` is checked only when the earlier branches did not match
`for` vs `while`	`for` suits a known number of repetitions; `while` repeats while a condition holds
`continue` vs `break`	`continue` goes to the next pass; `break` leaves the loop
`break` vs `return`	`break` leaves a loop; `return` leaves a function and can send a value back
parameter vs argument	the name in the definition vs the value in the call
`print()` vs `return`	`print()` displays; `return` passes a value to the caller
local vs module scope	local names live inside one function call; module names live at the top of the script
condition vs exception	business rules use `if`; expected conversion failures use a specific `except`
helper vs coordinator	a helper does one job; `process_order()` and `main()` coordinate helpers

Check your understanding

1. For `945.0 >= 950.0`, what Boolean is produced and which branch runs?
2. Why can 1181.25 be misclassified if `>= 500.0` is checked before `>= 950.0`?
3. What does `return` exit that `break` does not?
4. Why does a function that only prints a subtotal give `None` to its caller?
5. Why can a variable created inside a function not be used outside it?
6. Why is `four` an exception case for a quantity, but `-2` an ordinary validation case?
7. When does the `else` block of a `try` run, and when does `finally` run?
8. What belongs in `process_order()`, and what belongs in `main()`?

What comes next

The program still handles one order at a time and forgets each order once it is done. Session 09 adds ways to keep many values together (lists and dictionaries) and to save them in files.

Appendix: Answers to the exercises

Part 2

a2_tiers.py

```
annual_spend = 12500.0

if annual_spend >= 20000:
    tier = "Gold"
elif annual_spend >= 10000:
    tier = "Silver"
else:
    tier = "Bronze"

print("Tier:", tier)
```

OUTPUT

```
Tier: Silver
```

With 25 000 the tier is Gold; with 4 000 it is Bronze.

a2_delivery.py

```
order_total = 450.0

if order_total >= 500:
    delivery_fee = 0.0
else:
    delivery_fee = 60.0

print("Amount due:", order_total + delivery_fee)
```

OUTPUT

```
Amount due: 510.0
```

Part 3

a3_savings.py

```python
goal = 10000.0
monthly_saving = 1500.0
savings = 0.0
months = 0

while savings < goal:
    savings = savings + monthly_saving
    months = months + 1
    print(f"Month {months}: R{savings:.2f}")

print(f"Goal reached in {months} months.")
```

OUTPUT

```
Month 1: R1500.00
Month 2: R3000.00
Month 3: R4500.00
Month 4: R6000.00
Month 5: R7500.00
Month 6: R9000.00
Month 7: R10500.00
Goal reached in 7 months.
```

a3_weekly.py

```python
total_sales = 0.0

for week in range(1, 5):
    sales = float(input(f"Sales for week {week}: "))
    total_sales = total_sales + sales

print(f"Total: R{total_sales:.2f}")
print(f"Average: R{total_sales / 4:.2f}")
```

TERMINAL

```
> python a3_weekly.py
Sales for week 1: 12000
Sales for week 2: 15500
Sales for week 3: 9800
Sales for week 4: 14200
Total: R51500.00
Average: R12875.00
```

Part 4

a4.py

```
def apply_discount(subtotal, discount_pct):
    discount_amount = subtotal * discount_pct / 100
    return subtotal - discount_amount

def delivery_fee(order_total, free_threshold=500.0):
    if order_total >= free_threshold:
        return 0.0
    return 60.0

print(apply_discount(1000.0, 10.0))
print(delivery_fee(620.0))
print(delivery_fee(450.0))
print(delivery_fee(450.0, 400.0))
```

OUTPUT

```
900.0
0.0
60.0
0.0
```

Part 5

a5.py

```
def clean_text(text):
    return text.strip()

def build_order_label(customer_name, product_name):
    clean_customer = clean_text(customer_name).title()
    clean_product = clean_text(product_name).title()
    return clean_customer + " - " + clean_product

print(build_order_label("  maya patel ", " analytics starter kit "))
```

OUTPUT

```
Maya Patel - Analytics Starter Kit
```

`build_order_label` calls `clean_text` twice, then uses `.title()` on each returned value.

Part 6

a6.py

```python
def read_percentage(prompt):
    while True:
        raw_value = input(prompt)
        try:
            value = float(raw_value)
        except ValueError:
            print("Please enter a number.")
            continue
        if value < 0:
            print("Percentage cannot be below 0.")
            continue
        elif value > 100:
            print("Percentage cannot be above 100.")
            continue
        return value

discount_pct = read_percentage("Discount %: ")
print("Accepted:", discount_pct)
```

TERMINAL

```
> python a6.py
Discount %: ten
Please enter a number.
Discount %: -5
Percentage cannot be below 0.
Discount %: 120
Percentage cannot be above 100.
Discount %: 10
Accepted: 10.0
```

`ten` takes the exception path. `-5` and `120` convert successfully and are rejected by ordinary conditions.

Part 7

C, E, D, A, B: constant, helper function, `process_order()`, `main()`, entry guard.

Part 8: completed sales_order_processor.py

sales_order_processor_solution.py

```python
# Sales Order Validation & Processing Program -- completed

# 1. Constants
LARGE_ORDER_THRESHOLD = 950.0


# 2. Input helpers
def read_required_text(prompt):
    while True:
        text = input(prompt).strip()
        if text == "":
            print("This field cannot be blank.")
            continue
        return text
```

```
def read_positive_number(prompt):
    while True:
        raw_value = input(prompt)
        try:
            value = float(raw_value)
        except ValueError:
            print("Please enter a number.")
            continue
        if value <= 0:
            print("Value must be greater than 0.")
            continue
        return value


def read_positive_int(prompt):
    while True:
        raw_value = input(prompt)
        try:
            value = int(raw_value)
        except ValueError:
            print("Please enter a whole number.")
            continue
        if value <= 0:
            print("Value must be greater than 0.")
            continue
        return value


def read_percentage(prompt):
    while True:
        raw_value = input(prompt)
        try:
            value = float(raw_value)
        except ValueError:
            print("Please enter a number.")
            continue
        if value < 0:
            print("Percentage cannot be below 0.")
            continue
        elif value > 100:
            print("Percentage cannot be above 100.")
            continue
        return value


# 3. Calculation helpers
def calculate_subtotal(unit_price, quantity):
    return unit_price * quantity


def apply_discount(subtotal, discount_pct):
    discount_amount = subtotal * discount_pct / 100
    return subtotal - discount_amount


def apply_tax(discounted_subtotal, tax_pct):
    tax_amount = discounted_subtotal * tax_pct / 100
    return discounted_subtotal + tax_amount
```

```
def calculate_final_total(unit_price, quantity, discount_pct, tax_pct):
    subtotal = calculate_subtotal(unit_price, quantity)
    discounted_subtotal = apply_discount(subtotal, discount_pct)
    return apply_tax(discounted_subtotal, tax_pct)


def classify_order(final_total, threshold=LARGE_ORDER_THRESHOLD):
    if final_total >= threshold:
        return "large"
    else:
        return "standard"


# 4. Display
def display_summary(customer_name, product_name, unit_price, quantity,
                    discount_pct, tax_pct, final_total, classification):
    print()
    print("--- Order Summary ---")
    print("Customer:", customer_name)
    print("Product:", product_name)
    print("Unit price:", unit_price)
    print("Quantity:", quantity)
    print("Discount %:", discount_pct)
    print("Tax %:", tax_pct)
    print("Final total:", final_total)
    print("Classification:", classification)


# 5. One complete order
def process_order():
    customer_name = read_required_text("Customer name: ")
    product_name = read_required_text("Product name: ")
    unit_price = read_positive_number("Unit price: ")
    quantity = read_positive_int("Quantity: ")
    discount_pct = read_percentage("Discount %: ")
    tax_pct = read_percentage("Tax %: ")

    final_total = calculate_final_total(unit_price, quantity, discount_pct, tax_pct)
    classification = classify_order(final_total)

    display_summary(customer_name, product_name, unit_price, quantity,
                    discount_pct, tax_pct, final_total, classification)


# 6. Program loop
def main():
    print("Sales Order Validation & Processing Program")

    while True:
        process_order()

        again = input("Process another order? (y/n): ").strip().lower()
        if again != "y":
            print("Program finished.")
            break


# 7. Entry guard
if __name__ == "__main__":
    main()
```